AMS 526: Numerical Linear Algebra for Computational and Data Sciences

Lecture 24: Biorthogonalization Methods; Other Krylov Subspace Methods; Preconditioners

Prof. Xiangmin Jiao

Stony Brook University

Outline

Tridiagonal Biorthogonalization

- Lanczos iteration may be viewed as *tridiagonal orthogonalization*. If it runs through n iterations for $A \in \mathbb{C}^{n\times n}$, then $A = QTQ^{\top}$
- If A is not Hermitian, we have to either give up the tridiagonal form as in reduction to Hessenberg form, or give up orthogonality
- Tridiagonal biorthogonalization method constructs

$$A = VTV^{-1},$$

  where V is nonsingular but generally not unitary
- Its adjoint is

$$A^* = WT^*W^{-1},$$

  where $W = V^{-*} = (V^*)^{-1} = (V^{-1})^*$
- *Biorthogonalization* refers to the fact that $W^*V = V^*W = I$

Matrix Form of Tridiagonal Biorthogonalization

- Starting from *arbitrary* nonzero v_1 and w_1 with $v_1^* w_1 = 1$, it constructs

$$\begin{aligned} AV_k &= V_{k+1}\tilde{T}_k \\ A^* W_k &= W_{k+1}\tilde{S}_k \\ V_k^* W_k &= W_k^* V_k = I \end{aligned}$$

- Let T_k and S_k be the $k \times k$ leading blocks of $\tilde{T}_k$ and $\tilde{S}_k$, respectively.

$$T_k = S_k^*$$

  since $W_k^* A V_k = T_k$ and $V_k^* A^* W_k = S_k$
- Tridiagonal biorthogonalization is also known as *nonsymmetric Lanczos iteration*

Nonsymmetric Lanczos Iteration

- Let $\tilde{T}_k = \begin{bmatrix} \alpha_1 & \gamma_1 & & & \\ \beta_1 & \alpha_2 & \gamma_2 & & \\ & \beta_2 & \alpha_3 & \ddots & \\ & & \ddots & \ddots & \gamma_{k-1} \\ & & & \beta_{k-1} & \alpha_k \\ & & & & \beta_k \end{bmatrix}$
- Note that $\alpha_k = w_k^* A v_k$. Let $\beta_0 = \gamma_0 = 0$. Given v_k, w_k, α_k, γ_{k-1}, and β_{k-1}, we can determine $\beta_k v_{k+1}$ and $\bar{\gamma}_k w_{k+1}$ from

$$Av_k = \gamma_{k-1} v_{k-1} + \alpha_k v_k + \beta_k v_{k+1}$$
$$A^* w_k = \bar{\beta}_{k-1} w_{k-1} + \bar{\alpha}_k w_k + \bar{\gamma}_k w_{k+1}$$

- Choose β_k and γ_k arbitrarily under the constraint of $w_{k+1}^* v_{k+1} = 1$ (Note: the choices of β_k and γ_k are not unique)

Properties of Nonsymmetric Lanczos Iteration

- Nonsymmetric Lanczos iteration enjoys a three-term recurrence, instead of the k-term recurrence of Arnoldi iteration, at twice the cost of symmetric Lanczos iteration
- It essentially builds nonorthogonal bases $\{v_1, v_2, \ldots, v_k\}$ and $\{w_1, w_2, \ldots, w_k\}$ for Krylov subspaces $\mathcal{K}_k(A, v_1)$ and $\mathcal{K}_k(A^*, w_1)$, i.e.,

$$\mathcal{K}_k(A, v_1) = \text{span}\{v_1, Av_1, \ldots, A^{k-1}v_1\} = \text{span}\{v_1, v_2, \ldots, v_k\}$$
$$\mathcal{K}_k(A^*, w_1) = \text{span}\{w_1, A^*w_1, \ldots, (A^*)^{k-1}w_1\} = \text{span}\{w_1, w_2, \ldots, w_k\}$$

- Nonsymmetric Lanczos iteration may break down for some matrices if
  - $v_k = 0$ or $w_k = 0$, for which T is reducible, or
  - $v_k \neq 0$ and $w_k \neq 0$, but $w_k^* v_k = 0$ (this is more serious)
- Potential breakdowns pose robustness issues. This can be mitigated (but not fully resolved) by lookahead

BiConjugate Gradient Methods

- In CG, we find $x_k \in \mathcal{K}_k(A, v_1)$ such that

$$r_k \perp \mathcal{K}_k(A, v_1),$$

where $r_k = b - Ax_k$

- In BiConjugate Gradient, a.k.a. BCG or BiCG, we find $x_k \in \mathcal{K}_k(A, v_1)$ s.t.

$$r_k \perp \mathcal{K}_k(A^*, w_1)$$

- Similar to CG, let $x_k = V_k y_k$. Since $AV_k = V_{k+1}\tilde{T}_k$,

$$r_k = b - Ax_k = b - AV_k y_k = b - V_{k+1}\tilde{T}_k y_k$$

- Let $W_k^* r_k = 0$, then, $W_k^* b - W_k^* V_{k+1}\tilde{T}_k y_k = W_k^* b - T_k y_k = 0$
- At each step, we solve a $k \times k$ linear system

$$T_k y_k = W_k^* b$$

and then $x_k = V_k y_k$

Convergence of BiCG

- If A is HPD and $v_1 = w_1 = b/\|b\|$, then BiCG is equivalent to CG
- If A is Hermitian but indefinite, then BiCG is equivalent to SYMMLQ
- If A is non-Hermitian, the convergence of BiCG may be erratic and unpredictable, but it is more efficient than GMRES if it converges

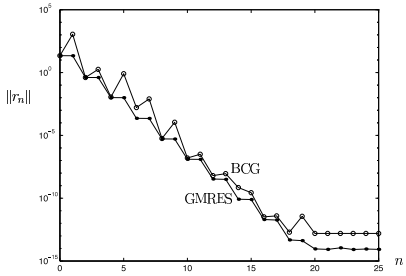

Figure 39.2. *Comparison of GMRES and BCG for the* 500×500 *matrix labeled* $\tau = 0.01$ *in Figure* 38.1, *but with the signs of the entries randomized.*

- BiCGSTAB smooths the convergence by resorting to GMRES sometimes, and it is transpose-free (van der Vorst, 1992)

Quasi-Minimal Residual Method

- QMR minimizes the residual in a different norm
- Similar to GMRES, let $x_k = V_k y_k$. Since $AV_k = V_{k+1}\tilde{T}_k$,

$$r_k = b - Ax_k = b - AV_k y_k = b - V_{k+1}\tilde{T}_k y_k$$

  and $W_{k+1}^* r_k = W_{k+1}^* b - \tilde{T}_k y_k$
- At each step, we solve a $(k+1) \times k$ least squares problem

$$\tilde{T}_k y_k \approx W_{k+1}^* b,$$

  which minimizes $\|W_{k+1}^* r_k\| = \|r_k\|_{W_{k+1}}$
- QMR is slower than BiCG, but it converges more smoothly
- QMR was developed by Freund and Nachtigal in 1991
- Transpose-Free QMR (TFQMR) was developed by Freund in 1993
- QMR/TFQMR use lookahead to mitigate (not resolve) breakdowns

Outline

Krylov Subspace Methods for Least Squares Problems

- CG on the Normal Equations (CGN)
  - Solve $A^*Ax = A^*b$ using Conjugate Gradients
  - Poor convergence due to squared condition number (i.e., $\kappa(A^*A) = \kappa(A)^2$)
  - One advantage is that it applies to least squares problems without modification
- Conjugate Gradients Squared (CGS)
  - Avoids multiplication by A^* in BCG, sometimes converges twice as fast as BCG

LSQR for Least Squares Problems

- It solves $Ax = b$, or minimizes $\|Ax - b\|^2$ or minimizes $\|Ax - b\|^2 + \lambda^2\|x\|^2$
- A may be square or rectangular (over-determined or under-determined), and may have any rank
- The method is based on the Golub-Kahan bidiagonalization process. It is algebraically equivalent to applying MINRES to the normal equation $(A^{\mathsf{T}}A + \lambda^2 I)x = A^{\mathsf{T}}b$, but has better numerical properties, especially if A is ill-conditioned
- LSQR reduces $\|r\|$ monotonically (where $r = b - Ax$ if $\lambda = 0$)
- References:
  - Paige, C. C. and M. A. Saunders, LSQR: An Algorithm for Sparse Linear Equations And Sparse Least Squares, *ACM Trans. Math. Soft.*, Vol.8, 1982, pp.43-71
  - http://www.stanford.edu/group/SOL/software/lsqr.html

Outline

Preconditioning

- Motivation: Convergence of iterative methods heavily depends on eigenvalues or singular values of the system
- The main idea of preconditioning is to introduce a nonsingular matrix M such that $M^{-1}A$ has better properties than A. Thereafter, solve

$$M^{-1}Ax = M^{-1}b,$$

  which has the same solution as $Ax = b$
- Criteria for M
  - "Good" approximation of A, depending on iterative solvers
  - Ease of inversion
- Typically, a preconditioner M is *good* if $M^{-1}A$ is not too far from normal and its eigenvalues are clustered

Left, Right, and Hermitian Preconditioners

- Left preconditioner: Left-multiply by M^{-1} and solve $M^{-1}Ax = M^{-1}b$
- Right preconditioner: Right-multiply by M^{-1} and solve $AM^{-1}y = b$ with $x = M^{-1}y$
- However, if A is Hermitian, $M^{-1}A$ or AM^{-1} breaks symmetry
- How to resolve this problem?

Left, Right, and Hermitian Preconditioners

- Left preconditioner: Left-multiply by M^{-1} and solve $M^{-1}Ax = M^{-1}b$
- Right preconditioner: Right-multiply by M^{-1} and solve $AM^{-1}y = b$ with $x = M^{-1}y$
- However, if A is Hermitian, $M^{-1}A$ or AM^{-1} breaks symmetry
- How to resolve this problem?
- Suppose M is Hermitian positive definite, with $M = CC^*$ for some C, then $Ax = b$ is equivalent to

$$\left[C^{-1}AC^{-*}\right](C^*x) = C^{-1}b,$$

  where $C^{-1}AC^{-*}$ is Hermitian positive definite, and it is similar to $C^{-*}C^{-1}A = M^{-1}A$ and has the same eigenvalues as $M^{-1}A$
- Example of $M = CC^*$ is Cholesky factorization $M = RR^*$, where R is upper triangular

Preconditioned Conjugate Gradient

- When preconditioning a symmetric matrix, use SPD matrix M, and $M = RR^{\mathsf{T}}$
- In practice, the algorithm can be organized so that only M^{-1} (instead of R^{-1}) appears

Algorithm: Preconditioned Conjugate Gradient Method

$x_0 = 0$, $r_0 = b$, $p_0 = M^{-1}r_0$, $z_0 = p_0$

for $k = 1, 2, 3, \ldots$

$\alpha_k = (r_{k-1}^{\mathsf{T}} z_{k-1})/(p_{k-1}^{\mathsf{T}} A p_{k-1})$	*step length*
$x_k = x_{k-1} + \alpha_k p_{k-1}$	*approximate solution*
$r_k = r_{k-1} - \alpha_k A p_{k-1}$	*residual*
$z_k = M^{-1} r_k$	*preconditioning*
$\beta_k = (r_k^{\mathsf{T}} z_k)/(r_{k-1}^{\mathsf{T}} z_{k-1})$	*improvement this step*
$p_k = z_k + \beta_k p_{k-1}$	*search direction*

Effective Preconditioners for CG

- SSOR Preconditioner
  - Simpler form: use a matrix splitting of the form $A = L + D + L^{\mathsf{T}}$ and take

$$M = (D + L)D^{-1}(D + L)^{\mathsf{T}}$$

  - More generally, introduce SSOR relaxation parameter ω, and take

$$M = \frac{1}{2-\omega}\left(\frac{1}{\omega}D + L\right)\left(\frac{1}{\omega}D\right)^{-1}\left(\frac{1}{\omega}D + L\right)^{\mathsf{T}}.$$

    With the optimal ω, $\mathrm{cond}(M^{-1}A) = \mathcal{O}\left(\sqrt{\mathrm{cond}(A)}\right)$, but determining the optimal ω is impractical
- Incomplete factorization
  - If $A = LL^{\mathsf{T}}$ were used as a preconditioner, then $\mathrm{cond}(M^{-1}A) = 1$, but it is impractical
  - Instead, compute an approximate factorization $A \approx \tilde{L}\tilde{L}^{\mathsf{T}}$, which omits all fill-ins or omits small fill-ins and use $M = \tilde{L}\tilde{L}^{\mathsf{T}}$ as a preconditioner

Other Commonly Used Preconditioners

- **Jacobi preconditioning**: $M = \text{diag}(A)$. Very simple and cheap, might improve certain problems but is usually insufficient
- **Block-Jacobi preconditioning**: Let M be composed of block-diagonals instead of diagonals
- **Multigrid** (**coarse-grid approximations**): For a PDE discretized on a grid, a preconditioner can be formed by transferring the solution to a coarser grid, solving a smaller problem, then transferring it back. This is sometimes the most efficient approach if applicable

Software Tools for Preconditioned Krylov Subspace Methods

- **PETSc (Portable, Extensible Toolkit for Scientific Computation)**: Offers robust support for Krylov subspace methods and various preconditioners, suitable for parallel computations.
- **Trilinos and Belos**: Trilinos includes extensive support for preconditioned Krylov solvers, applicable to large-scale scientific problems. Belos, part of Trilinos, provides a flexible package for iterative solvers, including various Krylov methods.
- **SciPy (Python) and MATLAB**: SciPy offers an implementation of GMRES and other Krylov methods, suitable for Python-based environments but may lack advanced features. MATLAB provides built-in functions for various Krylov subspace methods and is known for its user-friendly interface, though it may not be optimal for extremely large-scale problems.
- **ITSOL (Iterative Solvers Library)**: Developed by Yousef Saad, ITSOL is particularly useful as a teaching and learning tool.

Further Reading on Preconditioned Krylov Subspace Methods

- Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems* (2nd ed.). SIAM. [Provides an in-depth look into iterative methods, including Krylov subspace techniques.]
- Barrett, R. et al. (1994). *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. SIAM. [Focuses on various iterative methods for linear systems.]
- H. A. van der Vorst. (2003). *Iterative Krylov Methods for Large Linear Systems*. Cambridge University Press. [A comprehensive text on Krylov subspace methods for large linear systems.]
- Greenbaum, A. (1997). *Iterative Methods for Solving Linear Systems*. SIAM. [Covers a range of iterative methods for linear systems.]